

physics modelling agent

first prototype

Janis Jacobi
Niels Schröter

20th May, 2026

outline

- 1 goal of the project & aim of this prototype
- 2 architecture
 - constraints
 - crewai
- 3 example workflow
 - input
 - the Edelstein effect
 - pipeline
 - agent & task configuration
- 4 results
 - parameter choice
 - insights
- 5 summary

the general goal

goal

The “physics modelling agent” takes a research request as input and suggest a model which answers the research question

how to accomplish that

Find relevant sources

Extract important info from the sources

describe a model to answer the question

Create a numerical implementation

this prototype

what matches the final goal

- automatically finds real sources
- uses the information from these sources
- describes a reasonable model
- produces easy to execute code

what is still missing

- the python file does not execute right away
- the units of the implementation are wrong
- no bench marking yet
- no feedback inside the pipeline yet
- parameters have not been tweaked yet

architectural foundations

constraints

- only use open source software
- try to use only free software (no paid subscriptions)

design choices

- use SAIA from GWDG[7]
 - we have a valid API key for research purposes [6]
- use an existing framework
 - do not re-implement the entire logic for how to orchestrate different AI's
 - modify the parameters of existing frameworks to get the desired results



Figure: crewai logo [3]

crewai: core components

Agents[1]

accesses a LLM to work on a given topic

- accesses the LLM via specific object (allows to use SAIA or any other provider, specifies which model to use)
- behaviour can be defined via parameters (name, goal, backstory, temperature)
- tools can be added to allow advanced abilities (web search, code execution)

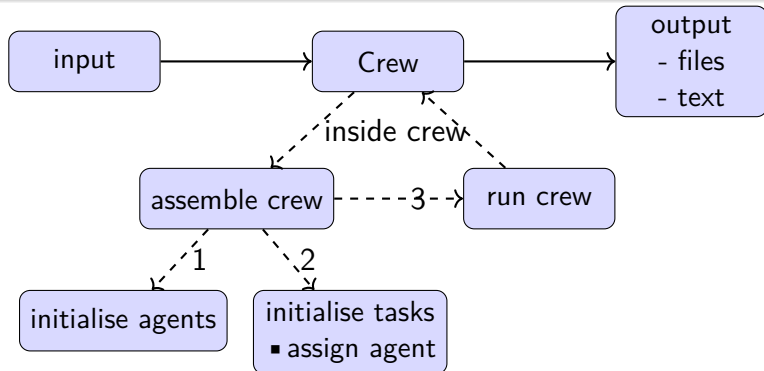
Tasks[4]

describe a concrete action to execute

- has task description and expected output parameters
- gets an agent object assigned (execution merges agent and task description)

Crew[2]

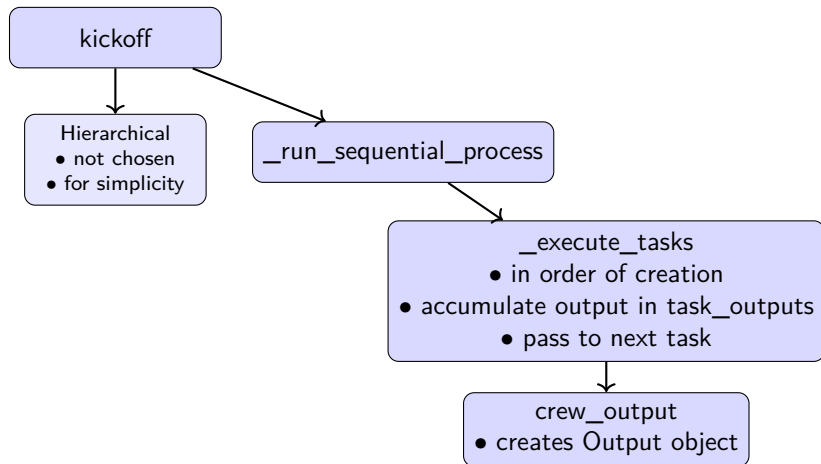
this puts it all together and orchestrates order of task executions



crews internal logic

upon creation of “crew” object:

- creates agent and task objects from config-files



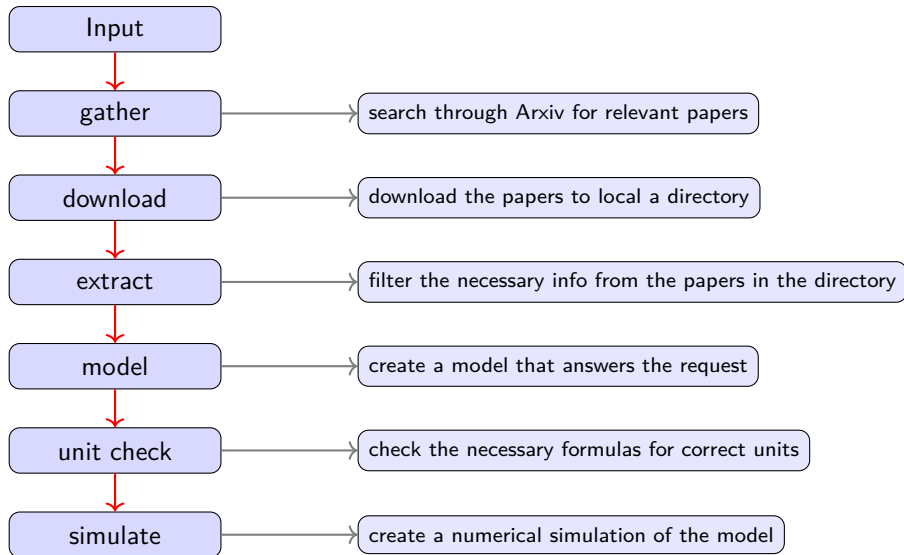
example: input/modelling request

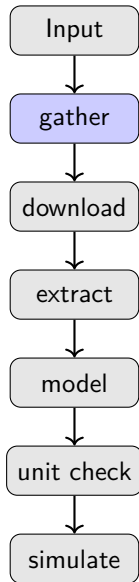
input

aim : “Calculate the Edelstein effect for a Rashba fermion (at the Gamma point of the Brillouin zone). Compute the magnetization magnitude and direction of different directions and magnitudes of the applied electric field. Consider how the result depends on relevant parameters of the model (e.g. chirality, fermi velocity, spin-orbit coupling strength) and make explicit graphics.”

topic : Edelstein-Effect

pipeline





agent: find relevant scientific papers

- temperature = 0.0; no creativity
- llm-model = qwen3.5-122b-a10b[10]; large inputs
- tools = ArxivPaperTool (searches through Arxiv)

task: list sources containing relevant information

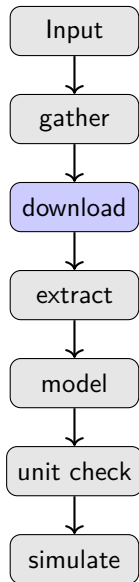
- output_file: create a markdown file

1. Edelstein Effect in Isotropic and Anisotropic Rashba Models

- **ArxivID:** 2503.20712
- **Authors:** Irene Gaiardoni, Mattia Trama, Alfonso Maiellaro, Claudio Guarcello, Francesco Romeo, Roberta Citro
- **Date:** 2025-03-26
- **URL:** <https://arxiv.org/pdf/2503.20712v1>
- **Summary:** We investigate spin-to-charge conversion via the Edelstein effect in a 2D Rashba electron gas using the semiclassical Boltzmann approach. We analyze the magnetization arising from the direct Edelstein effect, taking into account an anisotropic Rashba model. We study how this effect depends on the effective mass anisotropy, Fermi velocity, and other relevant parameters. This paper directly addresses the calculation of Edelstein effect for Rashba fermions and the dependence on model parameters.

Figure: source gatherer example output

download



agent: download papers to local directory

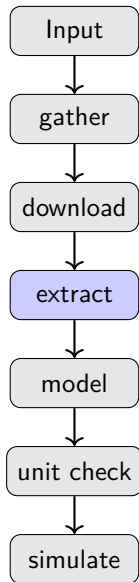
- temperature = 0.0; no creativity
- llm-model = qwen3.5-122b-a10b[10]; large inputs
- tools = ArxivDownloader (extends ArxivPaperTool)

task: download the papers; report issues

1. **Edelstein Effect in Isotropic and Anisotropic Rashba Models** (arXiv:2503.20712)
 - Authors: Irene Gaiardoni, Mattia Trama, et al.
 - ✓ Downloaded
2. **Theory of the nonlinear Rashba-Edelstein effect** (arXiv:1506.08330)
 - Authors: Giovanni Vignale, I. V. Tokatly
 - ✓ Downloaded
3. **Boltzmann theory of the Inverse Edelstein effect in a two-dimensional Rashba gas** (arXiv:2601.02473)
 - Authors: Irene Gaiardoni, Mattia Trama, et al.
 - ✓ Downloaded
4. **Spin and orbital Edelstein effect in a bilayer system with Rashba interaction** (arXiv:2307.02872)
 - Authors: Sergio Leiva M., Jürgen Henk, et al.
 - ✓ Downloaded
5. **Enhanced Edelstein effect and interdimensional effects in an electron gas with Rashba spin-orbit coupling interface** (arXiv:1912.01804)
 - Authors: A. C. Zulkoskey, R. Dick, K. Tanaka
 - ✓ Downloaded

Figure: download report

extract information I



agent: extract information using the downloaded papers

- temperature = 0.0; no creativity
- llm-model = qwen3.5-122b-a10b[10]; large inputs
- tools = PDFReader (reading pdf from local dir)

task: present info that allows building the model

1. System Hamiltonian

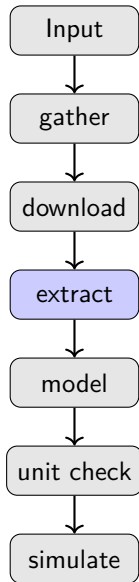
The system is described by a 2D electron gas with Rashba spin-orbit coupling (SOC). The Hamiltonian is given by:

$$\hat{H} = \frac{\hbar^2 k^2}{2m} + \alpha_R (\hat{z} \times \mathbf{k}) \cdot \boldsymbol{\sigma}$$

- **Source:** "Edelstein Effect in Isotropic and Anisotropic Rashba Models" (arXiv:2503.20712), Equation (1).
- **Parameters:**
 - m : Effective carrier mass.
 - α_R : Rashba spin-orbit coupling strength.
 - $\mathbf{k} = (k_x, k_y)$: Quasi-momentum.
 - $\boldsymbol{\sigma} = (\sigma_x, \sigma_y, \sigma_z)$: Vector of Pauli matrices.
 - $\hat{z}$: Unit vector perpendicular to the 2D plane.

Figure: information extractor example output

extract information II



other key findings:

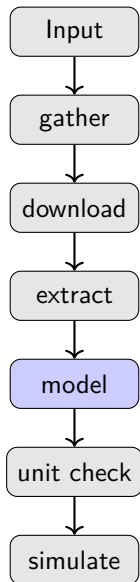
- Energy dispersion: $\epsilon_k^\nu = \frac{k^2}{2m} + \nu k \alpha$ mit $\nu = \pm 1$
- High-Density-Regime: ($\mu \geq 0$):
 - $k_F^\nu = -\nu k_0 + \sqrt{k_0^2 + 2m\mu}$
 - $M_y = \frac{\mu_b |e| \tau}{2\pi} m \alpha [\hat{z} \times \mathbf{E}]_y$
- Low-Density-Regime: ($\mu < 0$)
 - $k_F^\eta = k_0 - \eta \sqrt{k_0^2 + 2m\mu}$
 - $M_y = \frac{\mu_b |e| \tau}{2\pi} \sqrt{m^2 \alpha^2 + 2m E_F} [\hat{z} \times \mathbf{E}]_y$
- parameters:
 - chirality ν , in Low-Density-Regime also transport chirality η
 - Fermi velocity v_F , dependency implicitly via α and m
 - Rashba coupling α , for $\alpha \rightarrow 0$ Edelstein effect vanishes
 - effective mass m , in High-Density-Regime proportional to Magnetization M

model

agent: build a model that solves the problem

- temperature = 0.0; for reproducibility
- llm-model = deepseek-r1-distill-llama-70b[9]; reasoning

task: mathematical description with textual explanation
the model reproduces the formulas from the extracted information as seen before



- **Spin-Orbit Coupling Strength** (α_R): Magnetization scales linearly with α_R in HDR.
- **Fermi Velocity** (v_F): Higher v_F increases the susceptibility.
- **Chirality**: Determines the sign of magnetization.
- **Electric Field Direction**: Magnetization direction rotates 90° relative to $\mathbf{E}$.

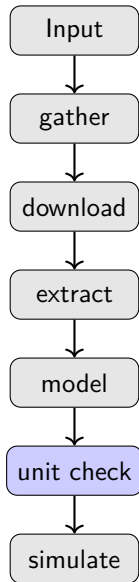
Figure: expected parameter dependencies

Explicit graphics should include:

1. Magnetization magnitude vs. electric field magnitude.
2. Magnetization direction vs. electric field direction.
3. Parameter dependence plots for α_R , E_F , and anisotropy ratios.

Figure: planning: which plots should be created

unit check



agent: check formulas and units

- temperature = 0.0; no creativity
- llm-model = deepseek-r1-distill-llama-70b[9]
- use previous work (no tools)

task: check the units of the model; correct them if necessary

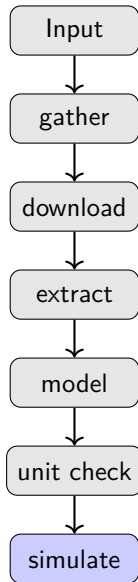
- $\mu_B \cdot e \cdot \tau \cdot m^* \cdot \alpha_R \cdot E_x$ has units:
 - $[A \cdot m^2] \cdot [C] \cdot [s] \cdot [kg] \cdot [E \cdot L] \cdot [kg \cdot m / (C \cdot s^3)]$.
 - Simplifies to $[A \cdot m^2 \cdot C \cdot s \cdot kg \cdot E \cdot L \cdot kg \cdot m / (C \cdot s^3)]$.
 - Further simplification gives $[A \cdot m^2 \cdot kg \cdot E \cdot L \cdot kg \cdot m / (s^2)]$.
 - Since energy $[E]$ has units of $[kg \cdot m^2 / s^2]$, substituting:
 - $[A \cdot m^2 \cdot kg \cdot kg \cdot m^2 / s^2 \cdot L \cdot kg \cdot m / (s^2)] = [A \cdot m^2 \cdot kg^3 \cdot m^3 / (s^4)]$.
 - This does not match the expected units of magnetization $[A/m]$.

- **Effective Mass (m^*):** $0.1 \times m_e$.
- **Rashba Parameter (α_R):** $1 \times 10^{-11} \text{ eV} \cdot m$.
- **Relaxation Time (τ):** $1 \times 10^{-12} \text{ s}$.
- **Fermi Energy (E_F):** $1 \times 10^{-2} \text{ eV}$.
- **Electric Field (E_x):** $1 \times 10^6 \text{ V/m}$.

Figure: unit checks: the unit of the magnetization does not match A m

Figure: starting parameters

simulation I



agent: create a numerical implementation of the model

- temperature = 0.1; even less creative
- llm-model = devstral-2-123b-instruct-2512[8]; coding

task: create executable python code

```
Magnetization magnitude: 1.64e-78 A/m
Magnetization direction: [0. 1. 0.]
Adiabaticity parameter gamma: 1.45e+31
```

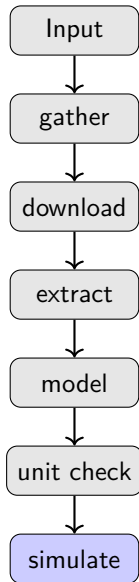
Figure: calculated magnetisation, wrong by many magnitudes:

$$M_y = \frac{\mu_B |e| \tau}{2\pi} n^* \alpha_R E_x$$

The calculation is wrong because of unit mismatches. This is caused by the incomplete formula.

The formula implementation sometimes drops the Bohr Magneton μ_B

simulation II



result: **executable code**

- the general relations match but their magnitudes are useless

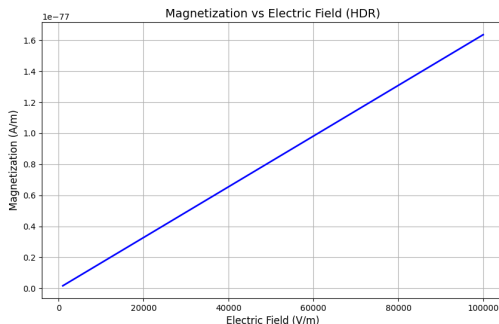


Figure: Magnetization vs. electric Field; linear dependency is expected

simulation III

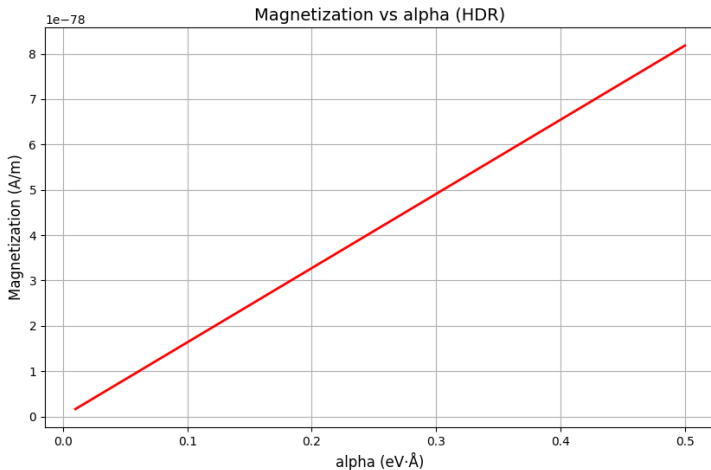
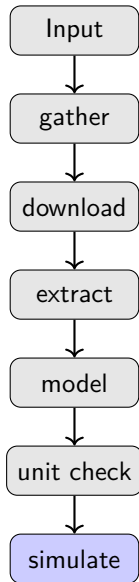
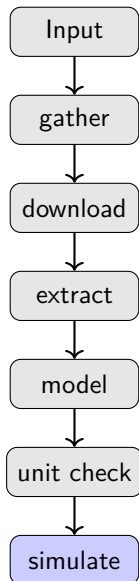


Figure: Magnetization vs. Rashba coupling; linear dependency is expected

simulation IV



Magnetization vs Electric Field Components (HDR)

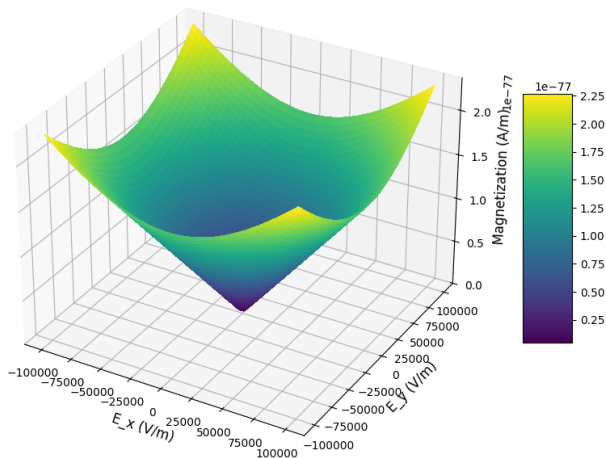


Figure: Magnetization vs. E-Field in 2D; the relation is correct for the total magnetization

parameter choice I: temperature & models

temperature - creativity of output (0.0 to 1.0)

- keep at 0.0 if the agent should only find or extract content
- → minimise hallucination
- allow some creativity for tasks requiring "thought": model making and coding
- **future**: test how temperature configuration affects the output

models

there is no general benchmark for all SAIA models

- for reasoning tasks use deep-seek models (benchmark: [9])
- for coding tasks use devstral (no scientific benchmark, github community "benchmark" at [8])
- for more general tasks use overall strong model qwen3.5 (benchmark [10])

parameter choice II: tools

tools enhance an agents capabilities

ArxivPaperTool

- crewai provided; finds papers and can read their metadata
- can not directly access the PDF contents or download them
- used for finding papers

ArxivDownloader

- custom tool to download papers from Arxiv; inherits from ArxivPaperTool

PDFReader

- custom tool, inherits from crewai's BaseTool
- allows agent to read PDF text (includes formulas); but not images

the key insights

working pipeline

- finding sources and getting information from Arxiv works
- the derived model appears to be reasonable
- the code is executable and the tendencies are correct

pipeline improvements

- unit check works okay but is unable to correct the formulas
- output varies over runs. More later.
- access a real RAG DB for better sources (better info base)
- allow direct code execution $\implies$ automate result assessment
- provide entry points into pipeline steps to rerun specific tasks
- include feedback loops (e.g. auto rerun agents if insufficient answer)

randomness

randomness

included in modern LLM's

- leads to different results for exact same setup
- try to eliminate randomness for reproducibility

effects of randomness

- the formula used for magnetization can vary over runs:

- $M_i = \chi_{ij} E_j$ (where

- $\chi_{ij} = -\chi_0 \sum_{\nu=\pm} \int d^2k < \sigma_y >_{\nu, \vec{k}} \delta(E_\nu(\vec{k}) - \mu) v_{\nu, x}(\vec{k})$

- or $M_y = \frac{\mu_B |e| \tau}{2\pi} \sqrt{k_0^2 + 2mE_f} [\hat{z} \times E]_z$

- sometimes additional plots are created (e.g. magnetisation over 2D electric field dependence)

reduce randomness

how to reduce randomness

temperature = 0 : this is the randomness parameter

verbose = False : suppress insertion of reasoning tokens

top_p & top_k = fixedNumber : removes sampling randomness

frequency_penalty & presence_penalty = 0 : removes LLM's bias against repetition

max_iter = 1 : number of reruns of an agent is nondeterministic

async_execution = False : execute tasks in specific order

effects

- reduced reasoning power (with verbose = 0): the input is not “interpreted” anymore, only what is explicitly demanded (e.g. plots are only generated for parameters mentioned in input)
- randomness could be needed when working with unsolved problems

ideas for research

agent & task definition

effect of goal, backstory, description, expected output, inputs phrasing

temperature

test how changes affect output

models

find better models for the specific tasks

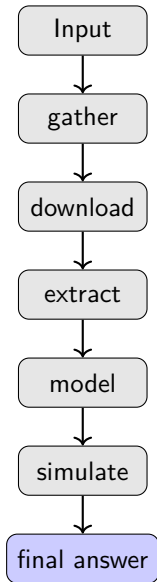
tools

- include a RAG DB and an open web search
- equip the simulator with a tool for python execution

benchmark the crew

use specific problem sets[5] from critpt[11]

benchmarking



currently trying to use critpt[11]

- replaced the “check agent” with an agent that provides a direct answer to the input.

not working yet

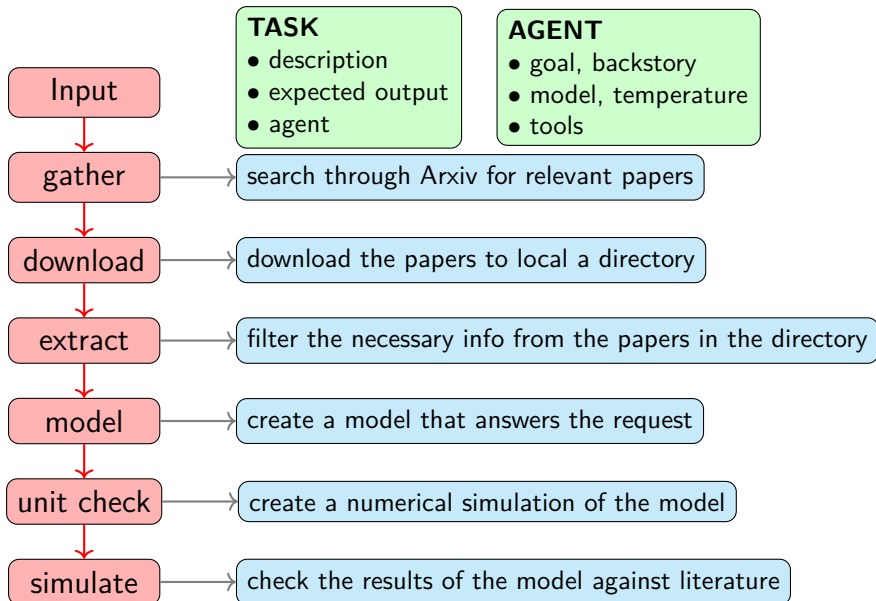
- the request to the evaluation server is rejected
- possible reasons (speculation): input format, API-key

improvements

optimize the pipeline for critpt input format:

- critpt provides context; add a context for agents
- grading on reasoning steps, tailor the crew to these
- concrete calculations expected

summary



- [1] crewai. *core.py*. 04/22/2026. URL: <https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/agent/core.py> (visited on 04/22/2026).
- [2] crewai. *crew.py*. 04/22/2026. URL: <https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/crew.py> (visited on 04/22/2026).
- [3] crewai. *crewAi*. 04/22/2026. URL: <https://crewai.com/> (visited on 04/22/2026).
- [4] crewai. *task.py*. 04/22/2026. URL: <https://github.com/crewAIInc/crewAI/blob/main/lib/crewai/src/crewai/task.py> (visited on 04/22/2026).

- [5] critPt. *challenges*. 04/22/2026. URL:
https://github.com/CritPt-Benchmark/CritPt/tree/main/data/public_test_challenges
(visited on 04/22/2026).
- [6] Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen. *AcademicCloud*. 04/21/2026. URL:
<https://id.academiccloud.de> (visited on 04/22/2026).
- [7] Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen. *SAIA*. 04/21/2026. URL:
<https://docs.hpc.gwdg.de/services/ai-services/saia/index.html> (visited on 04/22/2026).

- [8] Hugging Face. *Devstral 2 123B Instruct 2512*. 04/22/2026. URL: <https://huggingface.co/mistralai/Devstral-2-123B-Instruct-2512> (visited on 04/22/2026).
- [9] Daya Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning.” In: *Nature* 645.8081 (2025), pp. 633–638. ISSN: 1476-4687. DOI: [10.1038/s41586-025-09422-z](https://doi.org/10.1038/s41586-025-09422-z). URL: <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [10] Qwen Team. *Qwen3.5-Omni Technical Report*. 2026. arXiv: 2604.15804 [cs.CL]. URL: <https://arxiv.org/abs/2604.15804>.

- [11] Minhui Zhu et al. *Probing the Critical Point (CritPt) of AI Reasoning: a Frontier Physics Research Benchmark*. 2025. arXiv: 2509.26574 [cs.AI]. URL: <https://arxiv.org/abs/2509.26574>.

AI usage

The drawings were created with help of [microsoft copilot](#). I take full responsibility for the figures contents.